

Assignment 3

Vector Addition

Name: Arnav Singhal

MIS: 612203012

Div: 1; Batch: B3

1 Introduction

This assignment implements vector addition on the GPU to understand how CUDA organizes parallel work into a grid of thread blocks. Two integer arrays of size $N = 32000$ are added element-wise on the device, using a block size of $M = 32$ threads. The program was compiled with `nvcc -arch=sm_75` and executed on a Google Colab T4 GPU runtime.

2 The Program: Vector Addition

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

#define N 32000
#define M 32

__global__ void add(int *a, int *b, int *c, int n) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    if(index < n)
        c[index] = a[index] + b[index];
}

void random_ints(int *a, int size) {
    for(int i = 0; i < size; i++) {
        a[i] = (rand() % 100) + 1;
    }
}

void print_array(int *a, int size) {
    for(int i = 0; i < size; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

int main(void) {
    int *a, *b, *c;
    int *d_a, *d_b, *d_c;
    int size = N * sizeof(int);

    srand(time(NULL));

    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);

    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    add<<<(N + M - 1) / M, M>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);
```

```

printf("a = ");
print_array(a, N);
printf("b = ");
print_array(b, N);
printf("c = ");
print_array(c, N);

free(a);
free(b);
free(c);

cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_c);

return 0;
}

```

```

nvcc -arch=sm_75 vector.cu -o vector

./vector
a = 35 95 88 52 48 29 13 41 74 78 41 36 5 24 37 54 29 2 60 66 45 51 74 52 100 82 33 47 77 68 8 11 62 47 63 62 76 27 54 1 57 94 36 13 69 24 66 49 25 78 14 21 80
b = 29 40 29 34 90 1 71 28 75 93 96 97 31 5 46 95 18 47 23 76 42 59 28 9 48 78 68 55 66 62 13 46 53 41 31 94 41 53 21 67 97 68 63 80 72 60 74 90 6 49 65 47 7 9
c = 64 135 117 86 138 30 84 69 149 171 137 133 36 29 83 149 47 49 83 142 87 110 102 61 148 160 181 102 143 130 21 57 115 88 94 156 117 80 75 68 154 162 99 93 1

```

Figure 1: Output showing the randomly generated arrays a and b, and the element-wise sum c.

3 Grid, Block & Thread

- `blockDim.x`: number of threads per block, set here by the launch configuration to $M = 32$.
- `gridDim.x`: number of blocks in the grid, computed as $(N + M - 1) / M$, which rounds up so every one of the $N = 32000$ elements is covered even when N is not an exact multiple of M .
- `blockIdx.x`: index of the current block within the grid, ranging from 0 to `gridDim.x - 1`.
- `threadIdx.x`: index of the current thread within its own block, ranging from 0 to `blockDim.x - 1`.
- `index = threadIdx.x + blockIdx.x * blockDim.x`: maps each thread to one unique, global element index across the entire array, combining its block's position with its position inside that block.
- `if(index < n)`: boundary check that discards the extra threads launched by the rounded-up grid size, preventing out-of-bounds access on the last block.

4 Conclusion

The vector addition kernel confirmed that indexing threads via `blockIdx.x` and `threadIdx.x` correctly maps each of the $N = 32000$ elements to exactly one GPU thread across $(N + M - 1) / M$ blocks of $M = 32$ threads each, with the boundary check ensuring threads beyond the array bounds do no work.